Standard Template Library

- What is it?
  - Collection of container types and algorithms supporting basic data structures
- What is a container?
  - A generic list representation allowing programmers to specify which types of elements their particular lists hold
    - Uses the C++ template mechanism
- Have we seen this library before?
  - String class is part of the STL

STL Container Classes

- Sequences
  - deque, list, and vector
    - Vector supports efficient random-access to elements
- Associative
  - map, set
- Adapters
  - priority_queue, queue, and stack

- Our Focus: vector, map

Vector Class Properties

- Provides list representation comparable in efficiency to arrays
- Efficient subscripting is possible
  - Indices are in the range 0 … size of list - 1
- List size is dynamic
  - Can add items as we need them
- Index checking is possible
  - Through a member function
- Iterators
  - Efficient sequential access

Construction

- Basic construction

```
vector<T> List;
```

Container name

Base element type

- Example

```
vector<int> A;              // 0 ints
vector<float> B;            // 0 floats
vector<Rational> C; // 0 Rationals
```

- Template classes
  - Classes which can be instantiated with a specified type
  - How to define such classes? See chapter 14 of Cohoon, won't be covered in this course

Some Vector Constructors

- `vector<T>()`
  - creates a vector of zero length
- `vector<T>(int n)`
  - creates a vector of length `n`. Elements not initialized or default init.
  - E.g.

```
int n = PromptAndRead();
vector<int> D(n);          // n ints, uninitialized
vector<Rational> R(5);     // 5 rationals, default
                           // initialization
```

- `vector<T>(int n, T val)`
  - *Explicit* constructor creates a vector of length `n` with each element initialized to `val`
  - E.g.

```
vector<int> E(n, 3);       // n ints initialized to 3
Rational r(2,3);
vector<Rational> S(5, r);
```

Example

```
#include <vector>
#include <iostream>
using namespace std;
int main() {
   vector<int> A(4, 0); // A: 0 0 0 0
   A.resize(8, 2);      // A: 0 0 0 0 2 2 2 2
   vector<int> B(3, 1); // B: 1 1 1
   for (int i = 0; i < B.size(); ++i) {
       A[i] = B[i] + 2;
   }                    // A: 3 3 3 0 2 2 2 2
   A = B;               // A: 1 1 1
   return 0;
}
```

Vector Interface

- `size_type size()  // size_type : "same" as int`
  - Returns the number of elements in the vector

```
cout << A.size();      // display 3
```

- `bool empty()`
  - Returns true if there are no elements in the vector; otherwise, it returns false

```
if (A.empty()) {
    // ...
```

Vector Interface

- **Assignment**
  - Lhs becomes a copy of Rhs. "Shallow copy"

```
vector<int> A(4, 0); // A: 0 0 0 0
vector<int> B(3, 1); // B: 1 1 1
A = B;               // A: 1 1 1
```

- **Shallow copy: If A,B are vector<int*>, then only the pointers are copied, not what they point to.**
- "copy constructor" creates a vector that is a duplicate (shallow copy) of vector **v**.
  - Example: `vector<int> C(B); // C: 1 1 1`

Vector.at(i)

- **vector.at(i)**

  If **i** is in bounds, returns a reference to element **i** of the vector; otherwise, throws an exception

  - vector[i] behaves like array[i], doesn't check for out of bounds

```
vector<int> A(4, 0);      // A: 0 0 0 0

for (int i = 0; i <= A.size(); ++i) {
  A[i] = 3;
}                         // A: 3 3 3 3 ??

for (int i = 0; i <= A.size(); ++i) {
  A.at(i) = 3;
}                         // program terminates
                          // ("exception") when i is 4
```

Vector Interface

- `void resize(size_type s, T val = T())`
  - The number of elements in the vector is now **`s`**.
    - To achieve this size, elements are deleted or added as necessary
      - Deletions if any are performed at the end
      - Additions if any are performed at the end
      - New elements have value **`val`**

```
vector<int> A(4, 0); // A: 0 0 0 0
A.resize(8, 2);      // A: 0 0 0 0 2 2 2 2
A.resize(3,1);       // A: 0 0 0
```

Vector Interface

- `pop_back()`
  - Removes the last element of the vector
- `push_back(const T &val)`
  - Inserts a copy of `val` after the last element of the vector

Example

```
#include <iostream>
#include <vector>
using namespace std;

void GetV(vector<int> &A) {
   int Val;
   while (cin >> Val)
       A.push_back(Val); // A grows as required!
}
void PrintV(vector<int> &A){
  for(int i=0; i<A.size(); i++) cout << A[i];
}
int main(){
   vector<int> List;
   GetV(List); PrintV(List); // length not passed
}
```

Iterators

- Iterator is a pointer to an element
  - Really pointer abstraction
- Mechanism for sequentially accessing the elements in the list
  - Alternative to subscripting
- There is an iterator type for each kind of vector list
- Notes
  - Algorithm component of STL uses iterators
  - Code using iterators rather than subscripting can often be reused by other objects using different container representations

Vector Interface

- **`iterator begin()`**
  - Returns an iterator that points to the first element of the vector
- **`iterator end()`**
  - Returns an iterator that points to immediately *beyond* the last element of the vector

```
vector<int> C(4);  // C: 0 0 0 0
C[0] = 0; C[1] = 1; C[2] = 2; C[3] = 3;
vector<int>::iterator p = C.begin();
vector<int>::iterator q = C.end();
```

Iterators

- To avoid unwieldy syntax programmers often use typedef statements to create simple iterator type names

```
typedef vector<int>::iterator viiterator;

vector<int> C(4);  // C: 0 0 0 0
viiterator p = C.begin();
viiterator q = C.end();
```

Iterator Operators

- `*` dereferencing operator
  - Produces a reference to the object to which the iterator `p` points

    `*p`

- `++` point to next element in list
  - Iterator `p` now points to the element that followed the previous element to which p points

    `++p`

- `--` point to previous element in list
  - Iterator `p` now points to the element that preceded the previous element to which p points

    `--p`

- viiterator p = C.begin(), q = C.end();

```
 for(;p!=q; p++) {
   cout << *p << endl;
}
```

- for(int i=0; i < 10; i++) { cout << C[i] << endl;}
- int A[10];

```
for(int * p = A, i =0; i < 10; i++, p++) {
    cout << *p << endl;
}
```

Maps (not the geographic variety!)

Maps: Generalized Arrays

- Why should array indices be only integers?
- How about having an array of names, indexed by roll numbers, even though roll numbers are not integers?
- In some languages (but not in C++), you can do the following

```
array A[];
A["07100001"] = "Mohan";
A["07001101"] = "Meena";
foreach (a in A) {
    print a
}
```

Maps in C++

```
#include <iostream>
#include <map>
using namespace std;
int main () {
   map<string,string> A;
   A[“07100001”] = “Mohan”; // first element of A
   A[“07001101”] = “Meena”; // second element of A
   cout << A[“07100001”];  // prints “Mohan”.

   map<string,string>::iterator it;
   for ( it = A.begin(); it != A.end(); it++ )
       cout << (*it).first << " => " <<
               (*it).second << endl;
   return 0;
}
```

Maps in C++

```
map<string,string>::iterator it; /* declaration of it
          iterator: essentially pointer to elements of a map */

for ( it = A.begin(); it != A.end(); it++ )
    cout << (*it).first << " => " << (*it).second << endl;

/* A.begin: pointer to first element of A
   A.end: points after the last element of A
   begin and end are keywords.
   it++ : analogous to it = it->next */

/* (*it).first : index of whatever element it is pointing to,
   .second : value stored.  first, second are reserved words.
   Will print
              07100001 => Mohan
              07001101 => Meena
   */
```

Map usage

- Many uses
  - E.g. efficiently look up records based on key
    - (no need to keep sorted array + binary search, etc)
    - E.g. map<string, Entry*> // class Entry defined earlier
  - E.g. store property value pairs
    - Generalization of structs, where field names can be determined at run time
    - E.g. map<string, string> A[100];

```
A[0]["rollno"]="07100001";
A[0]["name"] = "Mohan";
A[1]["rollno"]="07001101";
A[1]["name"] = "Meena";
A[1]["phone"] = "9213123223";
```

Map Implementation

- Linked list implementation
  - Each node contains key and value
  - Search/insert straightforward but expensive (linear time)
- Idea: Hash Map implementation
  1. create an array of N linked lists
  2. given key K, compute a “hash” function which returns a value hash(K) in 0..N-1
  3. store key in linked list hash(K)
  4. when searching, look in linked list hash(K)
- Efficiency:
  - If N is chosen to be comparable to total number of elements in linked lists, each list expected to be small

Other Features of STL

- STL: Standard Template Library
- Other Containers
  - list<T>
  - queue<T>
  - priority_queue<compare_function, container>
  - bitset

More STL Algorithms (Ch. 9)

- Algorithms are mostly on containers/iterators
- Sorting
  - E.g. vector<int> v;
    sort(v.begin(), v.end());
  - or define your own comparison function
    bool cmp(int a, int b) { return a > b;}
    sort(v.begin(), v.end(), cmp);
- Set operations
  - intersection, union, difference
- search
- unique: remove duplicates
- permutations
- shuffle

Extra Slides

Example: vector of strings

- Consider

```
vector<string> A;
```

which is set in the following manner

```
A[0]: "A"
A[1]: "list"
A[2]: "of"
A[3]: "words"
A[4]: "to"
A[5]: "be"
A[6]: "read."
```

Counting o's

- The following counts number of o's within **A**

```
count = 0;

for (int i  = 0; i < A.size(); ++i) {

    for (int j = 0; A[i].size(); ++j) {

        if (A[i][j] == 'o') {

            ++count;

        }

    }

}
```

Size of **A**

Size of **A[i]**

To reference **j**th character of **A[i]** we need double subscripts

Two-Dimensional List

- E.g. 2 dimensional array: int A[n][n]
- For 2 dimensional vectors, consider definition

```
vector< vector<int> > A;
```

- Then

  `A` is a `vector< vector<int> >`
  - It is a vector of vectors

  `A[i]` is a `vector<int>`
  - `i` can vary from 0 to `A.size() - 1`

  `A[i][j]` is a `int`
  - `j` can vary from 0 to `A[i].size() - 1`
- **Can be passed to functions. Function can find row and column sizes as above.**

Exercises

- Write all programs you wrote using lists using vectors.
- Write a program that first reads in names of countries and their capitals from a file, and then reads country names from the keyboard and prints out their capitals. The program should stop when control-d is typed from the keyboard.
- Write a function which takes two matrices represented as vector of vectors, and returns their product. Check first that the matrices are compatible for multiplication.